

Perfectly Secure Network-Agnostic MPC Comes for Free

Xiaoyu Ji¹, Chen-Da Liu-Zhang², and Yifan Song³

¹ jixy23@mails.tsinghua.edu.cn, Tsinghua University

² chen-da.liuzhang@hslu.ch, Lucerne University of Applied Sciences and Arts

³ yfsong@mail.tsinghua.edu.cn, Tsinghua University and Shanghai Qi Zhi Institute

Abstract. Secure multiparty computation (MPC) allows a set of parties to jointly compute a function while keeping their inputs private. Classical MPC protocols assume either a synchronous or an asynchronous network. Synchronous protocols tolerate more corrupted parties but rely on a timing bound, while asynchronous protocols make no timing assumptions but handle fewer corruptions.

The network-agnostic model aims to combine the advantages of both. It requires security without knowing in advance whether the network is synchronous or asynchronous, guaranteeing resilience against up to t_s corruptions in the synchronous case and t_a corruptions in the asynchronous case. The optimal corruption threshold for perfect security has been established as $n = 2 \max(t_s, t_a) + \max(2t_a, t_s) + 1$, but prior work either falls short of this threshold or requires exponential local computation.

In this work, we present the first perfectly secure network-agnostic MPC protocol with polynomial communication and computation complexity under the optimal threshold. Our protocol achieves expected communication complexity $\mathcal{O}(|C|n + (D + C_I)n^2 + n^6) \log n$ bits for a circuit of size $|C|$ over a finite field $\mathbb{F}$ of size $\mathcal{O}(n)$, depth D , and input size C_I .

Our main technical contribution is a compiler that generates Beaver triples in the network-agnostic setting using synchronous and asynchronous triple-generation protocols in a black-box way. Beyond the cost of the underlying protocols, it only requires $\mathcal{O}(n^2)$ instances of network-agnostic Byzantine agreement.

1 Introduction

Secure multiparty computation (MPC) [Yao82, GMW87, BGW88, CCD88, RB89] enables a set of n parties to jointly compute a function on their private inputs while ensuring that any t corrupted parties learn nothing beyond what can be inferred from their own inputs and outputs.

Traditionally, the design of MPC protocols assumes a fixed communication model: either *synchronous* or *asynchronous*. In the synchronous model, parties proceed in lock-step rounds with synchronized clocks, and every message is guaranteed to be delivered within a known time bound Δ . This assumption enables strong security guarantees: statistically secure MPC can be achieved for $t < n/2$ given broadcast [RB89, DN07, BSFO12, GSZ20], and perfectly secure MPC is feasible for up to $t < n/3$ corrupted parties with guaranteed output delivery [BGW88, BH08]. The drawback, however, is brittleness: once messages are delayed beyond Δ , the protocol may completely lose security.

In contrast, the asynchronous model makes no assumptions on delivery time—messages may arrive after any finite delay, but honest parties’ messages are guaranteed to eventually be delivered. This makes asynchronous protocols robust to unpredictable networks, but comes with a cost in corruption tolerance: statistically secure asynchronous MPC requires $t < n/3$ [BKR94, CGHZ16, HNP05, Coh16, CHLZ21, GLZS24, BJK⁺25b, BJK⁺25a], and perfect security is possible only up to $t < n/4$ corruptions [BCG93, AAPP24]. In other words, synchronous protocols achieve higher resilience but rely on timing assumptions, while asynchronous protocols drop timing assumptions but tolerate fewer corruptions.

This trade-off naturally raises the question: can we design protocols that remain secure without knowing in advance whether the network is synchronous or asynchronous, and thereby inherit the best possible guarantees in either case? This is precisely the goal of the *network-agnostic model*. In this setting, parties are unaware of the underlying network type, yet the protocol guarantees security against up to t_s corruptions if the network is synchronous and up to t_a corruptions if the network is asynchronous. This model has received

increased attention in recent years and been considered in the study of multiple primitives including agreement [BKL19, DHLZ21, BKL21, ABKL22, GLZW22, GLZW23], secure-communication [DLZ23], distributed key generation [BCLZL23], and MPC [BZL20, DHLZ21, BCLZL23, ACC22, AC23, BC24, PP24].

The study of perfectly secure network-agnostic MPC was initiated by [ACC22], who achieved amortized $\mathcal{O}(n^5/t_a \log n)$ bits communication per multiplication gate with an additional $\mathcal{O}(n^7 \log n)$ additive overhead, under the corruption threshold $n = 3t_s + t_a + 1$. Later, [PP24] established the exact optimal threshold for perfectly secure network-agnostic MPC as $n = 2 \max(t_s, t_a) + \max(2t_a, t_s) + 1$, and presented a matching feasibility result. Their construction, however, relies on all parties agreeing on a clique of size $n - t_s$, which requires exponential local computation complexity. Consequently, no existing protocol achieves both polynomial communication and computation under the optimal corruption threshold. This motivates our work: constructing the first perfectly secure network-agnostic MPC with polynomial efficiency. At the same time, we are curious whether one can leverage existing synchronous and asynchronous MPC techniques and build a generic compiler for the network-agnostic setting, rather than designing protocols from scratch.

1.1 Our Contributions

In this work, we study the design of perfectly secure network-agnostic MPC with both polynomial communication and computation complexity, under the optimal corruption threshold $n = 2 \max(t_s, t_a) + \max(2t_a, t_s) + 1$ [PP24]. Our primary focus is on the amortized communication complexity required to evaluate each gate in the circuit.

For perfect security, existing works have already achieved amortized linear communication per multiplication gate in both the synchronous setting [BH08, GLS19, AAPP23] and the asynchronous setting [AAPP24, AAPS25], while keeping the computation complexity polynomial in the number of parties. This sharp contrast highlights a natural open question:

Can we also obtain a perfectly secure network-agnostic MPC with amortized linear communication and polynomial computation complexity?

We answer this question positively and obtain the following result.

Theorem 1. *Let n, t_s, t_a be positive integers such that $n = 2 \max(t_s, t_a) + \max(2t_a, t_s) + 1$. Let f be an n -ary function, there is a perfectly secure network-agnostic MPC protocol for f among n parties against a malicious adversary who controls up to t_s corrupted parties when the network is synchronous, and t_a corrupted parties when the network is asynchronous. For a function f with circuit size $|C|$ and circuit depth D , let C_I denote the input size, the achieved communication complexity is $\mathcal{O}((|C|n + (D + C_I)n^2 + n^6) \log n)$ bits in expectation.*

Note that under the optimal corruption threshold $n = 2 \max(t_s, t_a) + \max(2t_a, t_s) + 1$, the case $t_s \leq t_a$ reduces to $n = 4t_a + 1$. In this regime, one can simply run any perfectly secure asynchronous MPC protocol tolerating t_a corruptions. Hence, in the remainder of this work, we focus on the more challenging case $t_a < t_s$ and present our construction for this setting.

Main Technical Result. Following [PP24], we divide the construction of network-agnostic MPC into two phases: the offline phase and the online phase. In the offline phase, the parties prepare random Beaver triples [Bea92], which are then used to evaluate the circuit in the online phase. Unlike prior works [ACC22, PP24], our approach does not construct network-agnostic MPC by first building a network-agnostic secret-sharing scheme. Instead, we build the network-agnostic triple generation protocol by using a synchronous and asynchronous triple generation protocol in a black-box way. This yields the following theorem.

Theorem 2. *Let n, t_s, t_a be positive integers such that $n = 2 \max(t_s, t_a) + \max(2t_a, t_s) + 1$. There exists a perfectly secure network-agnostic triple generation protocol that generates N random Beaver triples with communication complexity $\mathcal{O}((Nn + n^6) \log n)$ bits.*

2 Technical Overview

In the following, let n denote the number of parties, t_s denote the synchronous corruption threshold, and t_a denote the asynchronous corruption threshold. We also assume that $t_a < t_s$ and set $n = 2t_s + \max(2t_a, t_s) + 1$.

In this section, we introduce the high-level idea of our network-agnostic triple generation protocol. Concretely, we construct a compiler that combines synchronous and asynchronous triple generation protocols to obtain a network-agnostic one. Recent works [DME24, JLZPS26] also propose compilers for constructing network-agnostic Byzantine agreement and MPC. However, their constructions focus on statistical security and rely on digital signatures, and therefore cannot be applied in the perfectly secure setting.

Throughout this subsection, when we say that a party broadcasts a message, we mean that it invokes an instance of network-agnostic reliable broadcast protocol with that message as input.

2.1 Safely Execute the Synchronous Protocol

We first attempt to let parties invoke the synchronous protocol in the network-agnostic setting. We note that the security of the synchronous protocol relies on two guarantees from the synchronous network: (1) all (honest) parties will receive their messages (from honest parties) within a fixed time bound in each round, and (2) all (honest) parties will proceed to the next round at the same time. However, if the network is asynchronous, we may face the following security and termination issues.

- An honest party A may fail to receive a message from another party B within the expected time bound. However, A cannot distinguish whether this is due to network asynchrony causing message delay, or because a corrupted B drops the message. As a result, A cannot determine whether it has already received the required messages from all honest parties in the current round, and thus may wait indefinitely.
- Even if an honest party receives all messages in a given round, it cannot guarantee that all other honest parties have done so. Since the security of a synchronous protocol may require that honest parties progress round-by-round in a coordinated manner, this prevents the party from safely moving to the next round.

To let parties make progress while maintaining the security, our idea is to design a protocol Π_{Channel} that helps parties to deliver their messages in each round of the synchronous protocol. If all parties acknowledge that all messages are delivered to each other, then they can safely proceed to the next round. Otherwise, if several parties' messages are delayed and cause the protocol stall, Π_{Channel} would help to identify parties that are responsible for this stall, and let all parties abort the protocol. With the help of Π_{Channel} , parties can safely execute a synchronous protocol in the network-agnostic setting, although they cannot guarantee the generation of triples so far. We first introduce the construction of Π_{Channel} as follows, then show how to utilize it in our compiler for triple generation in the next subsection.

Realization of Π_{Channel} . For each ordered pair (A, B) , we let the receiver A broadcast a confirmation message $(\text{Confirm}, A, B)$ if it received B 's message within the prescribed time bounds, and a conflict message $(\text{Conflict}, A, B)$ otherwise. To prevent from waiting indefinitely due to a corrupted receiver A never broadcasting either a confirmation or a conflict message, parties invoke a network-agnostic Byzantine agreement (BA) protocol with input 1 if they have received a broadcast from A regarding whether A has received B 's message within the prescribed time bound, and 0 otherwise.

- If BA outputs 1, then at least one honest party has received A 's broadcast. Honest parties wait until they receive either $(\text{Confirm}, A, B)$ or $(\text{Conflict}, A, B)$ from A .
- If BA outputs 0, then not all honest parties received A 's broadcast within the time bound. In this case, parties can know that if the network is synchronous, then A is a corrupted party. Then they treat $(\text{Conflict}, A, B)$ as the agreed outcome for this pair.

Note that with the help of the time bound, all honest parties eventually will have an input for each BA, thus guaranteeing the termination of these BA protocols. After parties reach an agreement on a confirmation or conflict outcome for every pair, if all outcomes are confirmations, then all required messages for the current

round have been received, and parties can safely advance to the next round of the synchronous protocol. Otherwise, if at least one outcome is a conflict, all parties can agree on the identity of parties (A, B) that are responsible for this failure. Refer to Section 4.2 for the detailed construction.

A Subtle Issue. In the synchronous case, when we say an honest party A can receive messages from another honest party B within prescribed time bound, we also require that both A and B should enter this round at a similar time. Since the time to enter the next round depends on the termination of BA or broadcast protocol in previous round, we need to ensure that all honest parties should terminate the BA or broadcast protocol at a similar time when the network is synchronous. This issue can be addressed with a modified version of Bracha’s reliable broadcast protocol [Bra84], and we show the details in Section 4.1.

2.2 High-level Idea of the Compiler

With the help of Π_{channel1} , we guarantee that once parties fail to make the progress in the synchronous protocol, they can identify a pair of parties (A, B) that are responsible for this failure. Then our first observation is that in a synchronous case, every honest party will receive all honest parties’ messages within the time bound. Consequently, for a pair of parties (A, B) that are responsible for the failure of the synchronous protocol, this ensures the following:

- If the network is synchronous, then at least one of A or B must be corrupted.
- If the network is asynchronous, it is possible that both A and B are honest.

Starting from this observation, we further note that if we directly remove both A and B upon failure, then among the remaining $n - 2$ parties there are at most $t_s - 1$ corrupted parties in the synchronous case and at most t_a corrupted parties in the asynchronous case. Moreover, the relation $n - 2 = 2(t_s - 1) + \max(2t_a, t_s - 1) + 1$ still holds. Therefore, the optimal corruption threshold is preserved for the remaining parties, and they can rerun the synchronous protocol.

If the protocol fails again, the parties eliminate another pair and repeat the procedure. Note that the number of corrupted parties will decrease by 1 in the synchronous case but 0 in the asynchronous case, so we will only repeats this elimination procedure for $t_s - t_a$ times to reduce the number of corrupt parties to t_a in both network scenarios. We note that in this case, since the number of remaining parties is at least

$$n - 2(t_s - t_a) = 2t_a + \max(2t_a, t_s) + 1 = \max(4t_a, 2t_a + t_s) + 1 \geq 4t_a + 1,$$

the remaining parties can safely switch to a perfectly secure asynchronous triple generation protocol that tolerates up to t_a corrupted parties.

To summarize, now parties can first attempt to safely invoke the synchronous protocol (with the help of Π_{channel1}) to generate triples. If they fail due to the delayed messages of several parties, they would directly eliminate these parties and let the remaining parties rerun the synchronous protocol. Upon eliminating a sufficient number of these parties, the remaining parties can safely switch to execute the asynchronous protocol to generate triples.

Further Optimization. If the remaining parties generate the full set of triples after each elimination, the procedure may repeat $t_s - t_a = \mathcal{O}(n)$ times, leading to at least quadratic amortized communication per triple. To achieve linear amortized cost, we adopt the party elimination framework of [HMP00]. Specifically, we divide triple generation into n segments, where each segment produces $1/n$ fraction of the triples.

However, a subtle issue arises: triples generated in different segments may have different degrees. Let $t_k = t_s - k$ denote the corruption threshold after eliminating $2k$ parties, and let $\mathcal{P}_k$ denote the remaining $n - 2k$ parties. If an arbitrary synchronous protocol is used, the parties in $\mathcal{P}_k$ generate triples of the form $([a]_{t_k}, [b]_{t_k}, [c]_{t_k})$, where each sharing has degree t_k . Thus, triples produced in different segments may have different degrees, which complicates parallel online evaluation and introduces additional overhead.

To resolve this, we use the protocol of [BH08]. Let n' denote the current number of parties, t' the number of corrupted parties, and d the desired degree of the triples. The protocol of [BH08] guarantees that the parties can generate degree- d triples as long as $d + 2t' < n'$.

In our setting, after k elimination steps, we have $n' = n - 2k$ and $t' = t_s - k$. We set the target degree to $d = t_s$. The condition holds as long as $k \leq t_s - t_a$:

$$d + 2t' = t_s + 2(t_s - k) < 2t_s + \max(2t_a, t_s) + 1 - 2k = n'.$$

Therefore, throughout the execution, the parties can consistently generate degree- t_s triples when invoking the synchronous triple generation protocols [BH08].

Remark 1. The protocol in [BH08] we used actually only achieves security with identifiable abort. That means when all parties terminate, they either output degree- t_s triples or agree on a pair of parties who are responsible for the failure of the protocol. The protocol ensures that at least one of these two parties is corrupted. Following a similar argument in Section 2.2, all parties can directly eliminate this pair of parties and let the remaining parties rerun the current segment.

Putting It All Together. We summarize the compiler as follows. All parties first try to invoke the synchronous protocol to generate degree- t_s triples. During this procedure, triple generation is divided into n segments. In each segment, the parties proceed as follows.

1. All parties execute a synchronous triple generation protocol with identifiable abort [BH08]. Messages in each round are delivered via Π_{Channel1} .
2. Upon terminating Π_{Channel1} , the parties either agree on a pair of parties to be eliminated or proceed to the next round. At the end of the segment:
 - If valid degree- t_s triples are produced, the parties proceed to the next segment.
 - Otherwise, the identified parties are eliminated and the segment is restarted.

During the execution of these n segments, if a total of $2(t_s - t_a)$ parties have been eliminated, all intermediate results from the synchronous phase are discarded. The remaining $4t_a + 1$ parties then invoke an asynchronous triple generation protocol to produce degree- t_a triples. Refer to Section 5.1 for the detailed construction of the compiler.

2.3 Online Error Correction Still Works

During the circuit evaluation, with the help of triples, parties only need to do public reconstruction. We use a modified version of the reconstruction protocol introduced in [DN07], and refer the reader to Section 5.2 for more details. It remains to show that the online error-correction procedure works in our setting to guarantee the correctness of the public reconstruction.

Suppose that the parties have terminated the offline phase and agreed on a subset of parties $\mathcal{P}'$ that hold shares of the generated triples. Let $|\mathcal{P}'| = n - 2k$ for some $k \in [0, t_s - t_a]$. If $k = t_s - t_a$, then the parties in $\mathcal{P}'$ hold degree- t_a triples and there are at most t_a corrupted parties. Otherwise, if $k < t_s - t_a$, the parties hold degree- t_s triples and the number of corrupted parties is at most $t_s - k$.

When $|\mathcal{P}'| = 4t_a + 1$ and $t = t_a$, correctness follows directly. We now consider the case $|\mathcal{P}'| = n_k = n - 2k$ for $k \in [0, t_s - t_a)$ and $t = t_s$. In this case, we have

$$\begin{aligned} \frac{n_k - t - 1}{2} &= \frac{n - 2k - t_s - 1}{2} = \frac{t_s + \max(2t_a, t_s) - 2k}{2} \\ &= \frac{1}{2} \max(2t_a + t_s - 2k, 2t_s - 2k) \geq t_s - k. \end{aligned}$$

Therefore, the reconstruction procedure can tolerate up to $t_s - k$ incorrect shares. Since there are at most $t_s - k$ corrupted parties in $\mathcal{P}'$, all incorrect values can be corrected, and the online phase succeeds. The same argument guarantees correctness of output reconstruction.

3 Preliminary

3.1 Model

We consider the universal composability (UC) framework [Can01] to prove the security of our protocols. Note that the standard UC framework considers an asynchronous network model where the adversary is allowed to drop messages. We are interested in 1) the synchronous model where parties have access to synchronized clocks and messages are delivered within a publicly known delay, and 2) the asynchronous setting with *eventual delivery*, where messages sent by honest parties are eventually delivered. To capture such models, several variants have been proposed in the literature for synchronous UC [KMTZ13, Can01, LZLM⁺19, BDD⁺21] where functionalities keep track of time, and asynchronous UC with eventual delivery [CGHZ16, LZLM⁺19, CFG⁺23]. Our protocols can be proven secure in any of those models. When modeling functionalities, we will use the term *request-based delayed output* [Coh16] to refer to eventual output delivery. Moreover, for functionalities in the synchronous setting, we use the term *time-bound delayed output* to ensure that the output is delivered within a prescribed time-bound, but the adversary may deliver the message earlier.

3.2 Ideal World

We consider the setting of multi-party computation with guaranteed output delivery (GOD). In this setting, we model the ideal MPC functionality $\mathcal{F}_{\text{sfe}}$ defined below with a flag **mode**, indicating whether the setting is synchronous or asynchronous. When the network is synchronous, the functionality will take into account all inputs from honest parties for the computation. When the network is asynchronous, only $n - t_s - t_a$ honest inputs are guaranteed to be taken into account, which is optimal as proven in [BZL20]. In both cases, the outputs are eventually delivered.

Functionality $\mathcal{F}_{\text{sfe}}$

$\mathcal{F}_{\text{sfe}}$ is parameterized by a set $\mathcal{P} = \{P_1, \dots, P_n\}$, an adversary $\mathcal{S}$, an n -party function $f : (\{0, 1\}^* \cup \{\perp\})^n \rightarrow \{0, 1\}^* \cup \{\perp\}$ and a flag **mode** $\in \{\text{synch}, \text{asynch}\}$. Let t_s (resp. t_a) be the corruption in the synchronous (resp. asynchronous) mode.

For each party P_i , initialize an input value $x^{(i)} = \perp$ and output value $y^{(i)} = \perp$.

Initialization

- 1: $\mathcal{F}_{\text{sfe}}$ receives the set Corr of corrupted parties from $\mathcal{S}$ and the network configuration **mode** $\in \{\text{synch}, \text{asynch}\}$. $\mathcal{F}_{\text{sfe}}$ checks that $|\text{Corr}| \leq t_s$ if **mode** = **synch** and $|\text{Corr}| \leq t_a$ if **mode** = **asynch**. If not, $\mathcal{F}_{\text{sfe}}$ gives up the security^a. Otherwise, $\mathcal{F}_{\text{sfe}}$ initializes **CoreSet** to be the set $\mathcal{H}$ of all honest parties.

Computation

- 1: Upon receiving an input (**Input**, v) from $P_i \in \mathcal{P}$, set $x^{(i)} = v$. Send a message (**Input**, P_i) to $\mathcal{S}$.
- 2: If $x^{(i)}$ has been set to a value different from $\perp$ for every honest $P_i \in \text{CoreSet}$, then compute $y = f(x^{(1)}, \dots, x^{(n)})$ (set $x^{(i)} = \perp$ for each $P_i \notin \text{CoreSet}$) and generate a request-based delayed output $y^{(i)} = y$ for every $P_i \in \mathcal{P}$.
- 3: Upon receiving an input **CoreSet'** from $\mathcal{S}$ and the output has not been delivered to any party, verify that **CoreSet'** is a subset of $\mathcal{P}$ of size at least $n - t_s$ (and if **mode** = **synch**, also check that **CoreSet'** contains all honest parties), else ignore the message. Then record **CoreSet** = **CoreSet'**. Finally, $\mathcal{F}_{\text{sfe}}$ re-executes Step 2 and updates all generated request-based delayed output according to the new **CoreSet**.

^a By giving up the security, $\mathcal{F}_{\text{sfe}}$ sends its view to $\mathcal{S}$ and allows $\mathcal{S}$ to arbitrarily decide the output of honest parties and the delivery time.

3.3 Real World

Network Model. In our protocols, parties have access to a complete network of secure point-to-point channels. Our protocols operate in two possible settings: synchronous or asynchronous.

In the synchronous setting, all parties have access to synchronized clocks and all messages are guaranteed to be delivered within some known upper bound delay Δ . Within Δ , the adversary can schedule the messages arbitrarily. In particular, the adversary is *rushing*, i.e., within the same round, the adversary is allowed to send its messages after seeing the honest parties' messages. Sometimes it is convenient to describe a protocol in rounds, where each round r refers to the interval of time $(r-1)\Delta$ to $r\Delta$. In such case, we say that a party receives a message in round r if it receives the message within that time interval. Moreover, we say a party sends a message in round r when it sends the message at the beginning of the round, i.e., at time $(r-1)\Delta$.

In the asynchronous setting, both assumptions above are removed. That is, parties do not have access to synchronized clocks (but they advance), and the adversary is allowed to arbitrarily schedule the delivery of the messages. However, we assume that all messages are eventually delivered (i.e., the adversary cannot drop messages).

Adversary Model. We consider a static adversary who corrupts parties in an arbitrary manner at the beginning of the protocol, i.e. active corruption. In the synchronous setting, the adversary can corrupt up to t_s parties. In the asynchronous setting, the adversary can corrupt up to t_a parties.

3.4 Network-Agnostic reliable broadcast

In a reliable broadcast (RBC) protocol, a sender P_s takes a message $m \in \mathcal{M}$ as input, where $\mathcal{M}$ is the message space which contains $\perp$. The protocol allows the sender to consistently distribute the message m among a set of parties. We recall the property-based definition of RBC protocol as follows.

Definition 1. Let t be the corruption threshold and Π be a protocol executed by parties $\mathcal{P} = \{P_1, \dots, P_n\}$, where a sender $P_s \in \mathcal{P}$ holds an input $m \in \mathcal{M}$, and all parties terminate upon getting output.

- ***t*-Validity:** If the sender P_s is honest, then all honest parties output m .
- ***t*-Consistency:** If an honest party terminates, then his output is a valid message $m' \in \mathcal{M}$, and all honest parties will eventually terminate with output m' .

If Π achieves t -validity and t -consistency, we say that Π is t -secure. A network-agnostic RBC protocol is t_s -secure in the synchronous setting and t_a -secure in the asynchronous setting. The authors in [PP24] give a construction of a network-agnostic RBC protocol with $\mathcal{O}(Ln^2)$ communication bits for broadcasting an L -bit message. We use Time_{bc} to denote the running time of Π when the dealer is honest and the network is synchronous.

3.5 Network-Agnostic Byzantine Agreement

In a Byzantine agreement (BA) protocol, each party P_i takes a message $m_i \in \mathcal{M}$ as input, where $\mathcal{M}$ is the message space which contains $\perp$. The protocol allows all parties to agree on a common message in $\mathcal{M}$. We recall the property-based definition of a BA protocol as follows.

Definition 2. Let t be the corruption threshold and Π be a protocol executed by parties $\mathcal{P} = \{P_1, \dots, P_n\}$, where each party initially holds an input $m_i \in \mathcal{M}$, and all parties terminate upon getting output.

- ***t*-Validity:** If all honest parties have the same input $m \in \mathcal{M}$, then all honest parties take m as output.
- ***t*-Consistency:** Honest parties do not obtain different outputs in $\mathcal{M}$.
- ***t*-Termination:** Suppose all honest parties participate in the protocol with their inputs in $\mathcal{M}$, and they never stop participating until termination. Then all honest parties terminate with a valid output in $\mathcal{M}$.

For a BA protocol Π , if Π achieves t -validity, t -consistency, and t -termination, we say Π is t -secure. We say Π is a network-agnostic BA protocol if Π is t_s -secure in the synchronous setting and t_a -secure in the asynchronous setting.

To instantiate a perfectly secure network-agnostic BA protocol, the authors in [PP24] give a construction, which invokes n instances of network-agnostic reliable broadcasts and an asynchronous BA protocol in a black-box way. By using the reliable broadcast protocol in [PP24] and asynchronous BA protocol in [AAPP24], it can be achieved with $\mathcal{O}(Ln^3 + n^4 \log n)$ bits of communication for L -bit agreement.

4 Secure Channel with Synchronous Identifiable Abort

4.1 Guarantee a Similar Termination Time for Agreement Protocol in the Synchronous Network

In our later construction, we require that all parties terminate the agreement protocol at a similar time when the network is synchronous. In this subsection, we describe how to ensure that all parties terminate an arbitrary network-agnostic agreement protocol Π at approximately the same time when executed over a synchronous network. We construct a modified protocol Π' as follows. Let Δ denote the maximum message delivery delay in the synchronous network. Then, in the synchronous setting, all parties are guaranteed to terminate Π' within a time difference of at most 2Δ .

Protocol Π'

Let Π denote a network-agnostic agreement protocol, which guarantees all parties terminate with the same output and is secure against $1/3$ fraction of corrupted parties in the synchronous setting and $1/4$ fraction of corrupted parties in the asynchronous setting.

- 1: All parties participate in Π after they learn their inputs, and we denote the agreement result of Π as m .
- 2: Upon receiving messages m in the following ways, send (Echo, m) to all parties:
 - Terminate Π with m .
 - The same (Echo, m) from $2t_s + 1$ distinct parties.
 - The same (Ready, m) from $t_s + 1$ distinct parties.
- 3: Upon receiving messages m in the following ways, send (Ready, m) to all parties:
 - The same (Echo, m) from $2t_s + 1$ distinct parties.
 - The same (Ready, m) from $t_s + 1$ distinct parties.
- 4: Upon receiving (Ready, m) from $2t_s + 1$ distinct parties, all parties terminate with output m .

We briefly explain why all parties will terminate within 2Δ time delay in the synchronous setting. Note that all parties may participate in the protocol at different times, and they are message-driven:

1. When the first honest party terminates, he has received (Ready, m) from $2t_s + 1$ distinct parties, which means that at least $t_s + 1$ honest parties have sent (Ready, m) to all parties.
2. In the following Δ time delay, the (Ready, m) sent from these honest parties will be delivered to all honest parties, and they will send (Ready, m) to all honest parties.
3. In the following Δ time delay, all honest parties are guaranteed to receive (Ready, m) from $2t_s + 1$ distinct parties, and they will terminate with the same message m .

Building on Π' and the network-agnostic RBC and BA protocols from [PP24], we obtain modified protocols Π'_{rbc} and Π'_{ba} that guarantee all parties terminate at approximately the same time. For an L -bit broadcast or agreement, the resulting communication complexity is $\mathcal{O}(Ln^2)$ bits and $\mathcal{O}(Ln^3 + n^4 \log n)$ bits, respectively.

4.2 Realization of Π_{Channel}

As we introduced in Section 2.1, we first consider a P2P message from a sender B to a receiver A . Let rid denote the round id, we assume that all honest parties enter the ridth round within a time difference of at

most Δ in the synchronous network. The construction of $\Pi_{\text{P2P-Send}}(A, B, \text{rid})$ is as follows. It guarantees that when all parties terminate $\Pi_{\text{P2P-Send}}(A, B, \text{rid})$, all parties will either agree that A receives messages from B or eliminate A and B (or only A).

Protocol $\Pi_{\text{P2P-Send}}(A, B, \text{rid})$

Sending Message

- 1: After B enters the rid -th round, B sends his message m to A .
- 2: After A enters the rid -th round, A waits for 2Δ time for the message m from B . If A receives the message from B within the time limit, A invokes Π'_{rbc} to broadcast a confirmation $(\text{Confirm}, A, B, \text{rid})$. Otherwise, A invokes Π'_{rbc} to broadcast a conflict $(\text{Conflict}, A, B, \text{rid})$.

Termination

- 1: Recall that Time_{bc} is the time required by Π'_{rbc} when the dealer is honest and the network is synchronous. All parties invoke Π'_{ba} and decide their input as follows: After entering the rid -th round, each party P_i waits for $4\Delta + \text{Time}_{\text{bc}}$ for the final broadcast message from A .
 - If the received broadcast message is in the form of $(\text{Confirm}, A, B, \text{rid})$ or $(\text{Conflict}, A, B, \text{rid})$, P_i sets his input of Π'_{ba} to be 1.
 - Otherwise P_i sets his input of Π'_{ba} to be 0.
- 2: All parties decide their output based on the result of Π'_{ba} and the final broadcast message from A .
 - If Π'_{ba} outputs 1 and the final broadcast message from A is $(\text{Confirm}, A, B, \text{rid})$, all parties output $(\text{Confirm}, A, B, \text{rid})$ and A outputs the received message m in addition.
 - If Π'_{ba} outputs 1 and the final broadcast message from A is $(\text{Conflict}, A, B, \text{rid})$, all parties output $(\text{Eliminate}, A, B, \text{rid})$.
 - Otherwise, all parties output $(\text{Eliminate}, A, \text{rid})$.

During the execution of synchronous protocol, parties may also need to broadcast the messages in some rounds. Although the broadcast channel can be realized by the P2P channel in the perfectly secure setting, we give a more direct realization of $\Pi_{\text{BC-Send}}$ to handle the broadcast message. The construction of $\Pi_{\text{BC-Send}}$ is similar to $\Pi_{\text{P2P-Send}}$, it takes the identity of party B and the round id rid as input, and B will reliably broadcast his messages at the beginning.

Protocol $\Pi_{\text{BC-Send}}(B, \text{rid})$

Sending Message

- 1: After B enters the rid -th round, B invokes Π'_{rbc} to broadcast his message m .
- 2: For each party A , after he enters the rid -th round, A waits for $\Delta + \text{Time}_{\text{bc}}$ time for the message m from B . If A receives the message from B within the time limit, A invokes Π'_{rbc} to broadcast a confirmation $(\text{Confirm}, A, B, \text{rid})$. Otherwise, A invokes Π'_{rbc} to broadcast a conflict $(\text{Conflict}, A, B, \text{rid})$.

Termination

The termination process is the same as $\Pi_{\text{P2P-Send}}$, except that each party P_i waits for $3\Delta + 2\text{Time}_{\text{bc}}$ for the final broadcast message from A .

Putting all it together. We construct the protocol Π_{Channel} from $\Pi_{\text{P2P-Send}}$ and $\Pi_{\text{BC-Send}}$. Each party takes as input a set $\mathcal{P}'$, which contains the identities of the parties that have been eliminated. For every remaining party $P_i \notin \mathcal{P}'$, it inputs a vector of messages and distributes them using $\Pi_{\text{P2P-Send}}$ and $\Pi_{\text{BC-Send}}$. Once all parties have terminated the executions of $\Pi_{\text{P2P-Send}}(A, B, \text{rid})$ and $\Pi_{\text{BC-Send}}(B, \text{rid})$ for $A, B \in \mathcal{P} \setminus \mathcal{P}'$, they either agree on new parties to be eliminated, or all alive parties in $\mathcal{P} \setminus \mathcal{P}'$ confirm that they have successfully received each other's messages.

Protocol Π_{Channel}

All parties take a set $\mathcal{P}'$ as input. Each party $P_i \notin \mathcal{P}'$ takes a vector of messages $(\text{rid}, m_0^{(i)}, \{m_j^{(i)}\}_{j \notin \mathcal{P}'})$ as input.

- 1: Each party $P_i \notin \mathcal{P}'$ takes $m_0^{(i)}$ as the broadcast message, and $m_j^{(i)}$ as the P2P message for each $P_j \notin \mathcal{P}'$. Then P_i invokes $\Pi_{\text{P2P-Send}}(A, B, \text{rid})$ for every $A, B \in \mathcal{P} \setminus \mathcal{P}'$, and invokes $\Pi_{\text{BC-Send}}(B, \text{rid})$ for every $B \in \mathcal{P} \setminus \mathcal{P}'$.
- 2: All parties initialize a set $\mathcal{W} = \emptyset$. For the output of each $\Pi_{\text{P2P-Send}}(A, B, \text{rid})$ and $\Pi_{\text{BC-Send}}(B, \text{rid})$:
 - If the output is in the form of $(\text{Eliminate}, A, B, \text{rid})$ or $(\text{Eliminate}, A, \text{rid})$, all parties add the identities of (A, B) or A to $\mathcal{W}$.
- 3: All parties terminate with $\mathcal{W}$. If $\mathcal{W} = \emptyset$, then each party $P_i \in \mathcal{P}'$ additionally terminates with $(\text{rid}, \mathcal{W}, \{(m_0^{(j)}, m_i^{(j)})\}_{j \notin \mathcal{P}'})$.

Optimization. In the above construction, all parties need to invoke Π'_{ba} in each call of $\Pi_{\text{P2P-Send}}$ and $\Pi_{\text{BC-Send}}$, which adds up to $O(n^2)$ calls of Π'_{ba} in each round. We show a simple optimization that reduces the number of calls of Π'_{ba} and Π'_{rbc} to $O(n)$.

In each round, for each party A , we only invoke a single call of Π_{ba} . Each party P_i sets his input to be 1 if he received the broadcast message from A in $\Pi_{\text{P2P-Send}}(A, B, \text{rid})$ and $\Pi_{\text{BC-Send}}(A, \text{rid})$ for all B within the time-bound, and sets his input to be 0 otherwise. Then P_i follows the termination phase of each $\Pi_{\text{P2P-Send}}(A, B, \text{rid})$ and $\Pi_{\text{BC-Send}}(A, \text{rid})$ to decide the output. We can similarly show that when the network is synchronous, an honest P_i will never set his input of Π_{ba} to be 0 for an honest A .

Based on this optimization, let $L_{\text{bc}}, L_{\text{P2P}}$ denote be the total broadcast message size and P2P message size, the communication complexity of Π_{Channel} is $\mathcal{O}(L_{\text{P2P}} + L_{\text{bc}}n^2 + n^5 \log n)$ bits.

5 Construction of Perfectly Secure Agnostic MPC

In this section, we present our construction of network-agnostic MPC. We begin with a perfectly secure agnostic triple-generation protocol, which serves as the foundation for realizing the full agnostic MPC protocol.

5.1 Network-Agnostic Triple Generation Compiler

In this subsection, we give the construction of our triple-generation protocol. Upon terminating the protocol, all parties will output a party set $\mathcal{P}'$, and parties in $\mathcal{P}'$ get their shares of random Beaver triples. If $|\mathcal{P}'| > 4t_a + 1$, then parties in $\mathcal{P}'$ get degree- t_s random Beaver triples. Otherwise, $|\mathcal{P}'| = 4t_a + 1$, and parties in $\mathcal{P}'$ get degree- t_a random Beaver triples.

To build this triple-generation protocol, we rely on two sub-protocols $\Pi_{\text{Triple-Id-Sync}}$ and $\Pi_{\text{Triple-Async}}$, which realize the functionalities $\mathcal{F}_{\text{Triple-Id-Sync}}$ and $\mathcal{F}_{\text{Triple-Async}}$ defined below, respectively. Let N denote the number of triples to be prepared, for the instantiation of these two functionalities:

- For $\mathcal{F}_{\text{Triple-Id-Sync}}$, we use the construction in [BH08] to build $\Pi_{\text{Triple-Id-Sync}}$, which securely computes $\mathcal{F}_{\text{Triple-Id-Sync}}$ against $1/3$ corruptions. The communication complexity is $\mathcal{O}((Nn + n^2) \log n)$ P2P communication bits plus $\mathcal{O}(\log N + n \log n)$ broadcast bits.⁴ The round complexity is $\mathcal{O}(1)$.
- For $\mathcal{F}_{\text{Triple-Async}}$, we can use the construction in [AAPS25] to build $\Pi_{\text{Triple-Async}}$, which securely computes $\mathcal{F}_{\text{Triple-Async}}$ against $1/4$ corruptions. The communication complexity is $\mathcal{O}((Nn + n^4) \log n)$ bits, and the expected round complexity is $\mathcal{O}(1)$.

⁴ We use a variant of [BH08], which uses $\mathcal{O}(n)$ instances of broadcast to replace the BA protocol. This helps to reduce the round complexity to a constant. Refer to Appendix A for more details.

Functionality $\mathcal{F}_{\text{Triple-Id-Sync}}$

$\mathcal{F}_{\text{Triple-Id-Sync}}$ runs with a party set $\mathcal{P}'$, and an adversary $\mathcal{S}$. Denote N as the number of random degree- t_s Beaver triples to be prepared.

- 1: Receive the set Corr of corrupted parties. Let $t := |\text{Corr}|$ and $n := |\mathcal{P}'|$, if $t_s + 2t < n$, $\mathcal{F}_{\text{Triple-Id-Sync}}$ proceeds. Otherwise, $\mathcal{F}_{\text{Triple-Id-Sync}}$ gives up security and sends all views to $\mathcal{S}$.
- 2: For all $i \in [N]$, randomly samples a_i, b_i, c_i such that $c_i = a_i \cdot b_i$.
- 3: For all $i \in [N]$, receive a set of shares $\{u_{i,j}, v_{i,j}, w_{i,j}\}_{j \in \text{Corr}}$ of corrupted parties from $\mathcal{S}$. Then sample three random degree- t_s Shamir sharings $([a_i]_{t_s}, [b_i]_{t_s}, [c_i]_{t_s})$ based on the shares of corrupted parties and the secrets a_i, b_i, c_i .
- 4: Send a request-based delayed output of shares of $\{([a_i]_{t_s}, [b_i]_{t_s}, [c_i]_{t_s})\}_{i=1}^N$ to all parties in $\mathcal{P}$.
 - Upon receiving a request $(\text{Conflict}, P_i, P_j)$ from $\mathcal{S}$, if the output of all parties has not been delivered and $P_i, P_j \in \mathcal{P}$, change it to $(\text{Conflict}, P_i, P_j)$. Otherwise, ignore this message.

Functionality $\mathcal{F}_{\text{Triple-Async}}$

$\mathcal{F}_{\text{Triple-Async}}$ runs with a party set $\mathcal{P}'$, and an adversary $\mathcal{S}$. Denote N as the number of random degree- t_a Beaver triples to be prepared.

- 1: Receive the set Corr of corrupted parties in $\mathcal{P}$. Let $t := |\text{Corr}|$ and $n := |\mathcal{P}'|$, if $t \leq t_a$ and $4t_a < n$, $\mathcal{F}_{\text{Triple-Async}}$ proceeds. Otherwise, $\mathcal{F}_{\text{Triple-Async}}$ gives up security and sends all views to $\mathcal{S}$.
- 2: For all $i \in [N]$, randomly samples a_i, b_i, c_i such that $c_i = a_i \cdot b_i$.
- 3: For all $i \in [N]$, receive a set of shares $\{u_{i,j}, v_{i,j}, w_{i,j}\}_{j \in \text{Corr}}$ of corrupted parties from $\mathcal{S}$. Then sample three random degree- t_a Shamir sharings $([a_i]_{t_a}, [b_i]_{t_a}, [c_i]_{t_a})$ based on the shares of corrupted parties and the secrets a_i, b_i, c_i .
- 4: Send a request-based delayed output of shares of $\{([a_i]_{t_a}, [b_i]_{t_a}, [c_i]_{t_a})\}_{i=1}^N$ to all parties in $\mathcal{P}$.

Then we give our construction of $\Pi_{\text{NA-Triple}}$ for triple generation. Let N be the number of triples to be prepared, the communication complexity is $\mathcal{O}((Nn + n^6) \log n)$ bits, and the round complexity is $\mathcal{O}(n)$.

Protocol $\Pi_{\text{NA-Triple}}$

Let $\Pi_{\text{Triple-Id-Sync}}$ be a synchronous Beaver triple generation protocol that securely computes $\mathcal{F}_{\text{Triple-Id-Sync}}$. Suppose R is the (upper bound of the) number of rounds required by $\Pi_{\text{Triple-Id-Sync}}$.

All parties define $\mathcal{R} = \{(\Pi_{\text{Triple-Id-Sync}}, j)\}_{j \in [R]}$ and initialize a set $\mathcal{W} = \emptyset$. They also take a public parameter N as input, which denotes the number of random Beaver triples to be prepared.

Regular Procedure

In this procedure, all parties aim to generate N degree- t_s random Beaver triples. They first divide the generation of N degree- t_s Beaver triples into n segments. In each segment, from $\text{rid} = 1$ to R , parties not in $\mathcal{W}$ execute the following steps to generate degree- t_s Beaver triples for all parties in $\mathcal{P} \setminus \mathcal{W}$.

- 1: Each party $P_i \notin \mathcal{W}$ follows the protocol $\Pi_{\text{Triple-Id-Sync}}$ to compute his rid -th round broadcast message $m_0^{(i)}$ and P2P messages $\{m_j^{(i)}\}_{j \notin \mathcal{W}}$, where each $m_j^{(i)}$ is the P2P message to P_j .
- 2: All parties invoke an instance of Π_{Channel} with input $\mathcal{W}$, each party $P_i \notin \mathcal{W}$ takes input $(\text{rid}, m_0^{(i)}, \{m_j^{(i)}\}_{j \notin \mathcal{W}})$. Upon receiving the output $\mathcal{W}'$ from Π_{Channel} , all parties compute $\mathcal{W} \leftarrow \mathcal{W} \cup \mathcal{W}'$.
- 3: All parties first check whether $|\mathcal{W}| \geq 2(t_s - t_a)$. If true, they move to the fallback procedure with the set $\mathcal{W}$. Otherwise, depending on the size of $\mathcal{W}'$, $\mathcal{W}$ and rid , all parties check whether $\mathcal{W}' = \emptyset$,
 - If true, then if $\text{rid} < R$, they move to the $(\text{rid} + 1)$ -th round of $\Pi_{\text{Triple-Id-Sync}}$. Otherwise, the $\text{rid} = R$, all parties compute their output following $\Pi_{\text{Triple-Id-Sync}}$ and move to the next segment. For the output of $\Pi_{\text{Triple-Id-Sync}}$,
 - If the output is $(\text{Conflict}, P_i, P_j)$, they add the identity of P_i, P_j to set $\mathcal{W}$ and execute the current segment again.
 - Otherwise, let $\mathcal{P}' = \mathcal{P} \setminus \mathcal{W}$, all parties in $\mathcal{P}'$ have generated valid degree- t_s Beaver triples. Then all parties terminate with $\mathcal{P}'$, parties in $\mathcal{P}'$ terminate with their degree- t_s Beaver triples.
 - Otherwise, they execute the current segment again.

Fallback Procedure

Let $\mathcal{P}'$ be the set of the first $4t_a + 1$ parties in $\mathcal{P} \setminus \mathcal{W}$, all parties in $\mathcal{P}'$ invoke $\Pi_{\text{Triple-Async}}$ to generate N degree- t_a Beaver triples for parties in $\mathcal{P}'$. Then all parties output the set $\mathcal{P}'$, parties in $\mathcal{P}'$ output their shares of degree- t_a Beaver triples.

Analysis of the Communication Complexity. The total communication costs of $\Pi_{\text{NA-Triple}}$ includes the costs of $\Pi_{\text{Triple-Id-Sync}}$, $\Pi_{\text{Triple-Async}}$ and Π_{Channel} . For each instance of $\Pi_{\text{Triple-Id-Sync}}$, the P2P communication bit is $\mathcal{O}((N + n^2) \log n)$ and the broadcast communication bit is $\mathcal{O}(\log N + n \log n)$, and there are $\mathcal{O}(1)$ rounds. Note that $n^2 \log N \leq N \log n^2$ when $N \geq n^2$, then the total cost of Π_{Channel} for each instance of $\Pi_{\text{Triple-Id-Sync}}$ is $\mathcal{O}((N + n^5) \log n)$ bits. Since the communication cost of $\Pi_{\text{Triple-Async}}$ is $\mathcal{O}((Nn + n^4) \log n)$ bits and there are $\mathcal{O}(n)$ instances of $\Pi_{\text{Triple-Id-Sync}}$, then the total communication complexity is $\mathcal{O}((Nn + n^6) \log n)$ bits.

5.2 Efficient Public Reconstruction

As we introduced in Section 2.3, we give the public reconstruction protocol Π_{PubRec} that will be used in the online phase. Let N be the number of sharings to be reconstructed, the communication complexity is $\mathcal{O}((Nn + n^2) \log n)$ bits.

Protocol Π_{PubRec}

All parties in $\mathcal{P}$ take a party set $\mathcal{P}'$ as input. Let N be the number of shares to be reconstructed, $\alpha_1, \dots, \alpha_n$ be deistinct field elements, t' denote the number of corrupted parties in $\mathcal{P}'$, t denote the degree of Shamir sharings to be reconstructed, and $n' = |\mathcal{P}'|$. The parameter t', t, n' satisfies the relation $(n' - t - 1)/2 \geq t'$. All parties take their shares of $[x_1]_t, \dots, [x_N]_t$ as inputs.

- 1: Divide $[x_1]_t, \dots, [x_N]_t$ into $N/(t+1)$ groups, each of size $t+1$. For each group, each party $P_i \in \mathcal{P}$ does the following.
 - (1). Let $[s^{(0)}]_t, \dots, [s^{(t)}]_t$ denote the degree- t Shamir secret sharings in this group. We define a degree- t polynomial $f(X)$ as follows.

$$f(X) = s^{(0)} + s^{(1)} \cdot X + \dots + s^{(t)} \cdot X^t$$

Then if $P_i \in \mathcal{P}'$, he sends his share of $[f(\alpha_j)]_t = [s^{(0)}]_t + [s^{(1)}]_t \cdot \alpha_j + \dots + [s^{(t)}]_t \cdot \alpha_j^t$ to each $P_j \in \mathcal{P}'$. Otherwise, P_i does nothing in this step.

- (2). Each party $P_i \in \mathcal{P}'$ uses online error correction on shares of $[f(\alpha_i)]_t$ received from parties in $\mathcal{P}'$ to reconstruct $f(\alpha_i)$. Then P_i sends $f(\alpha_i)$ to all parties in $\mathcal{P}$.
 - (3). Each party $P_i \in \mathcal{P}$ uses online error correction on $f(\alpha_j)$ received from $P_j \in \mathcal{P}'$ to reconstruct $f(X)$. Then P_i records $s^{(0)}, \dots, s^{(t)}$.
- 2: All parties in $\mathcal{P}$ output $x_1, \dots, x_N$.

5.3 Construction of Network-Agnostic MPC

We give the construction of $\Pi_{\text{NA-MPC}}$ to realize a perfectly secure agnostic MPC based on $\Pi_{\text{NA-Triple}}$. Let $|C|$ denote the circuit size, D denote the circuit depth, and C_I denote the input size. The achieved communication complexity is $\mathcal{O}((|C|n + n^6) \log n)$ bits in the offline phase and $\mathcal{O}((|C|n + (D + C_I)n^2 + n^5) \log n)$ in the online phase. The total expected round complexity is $\mathcal{O}(D + n)$.

A subtle issue in the input phase is that the parties not only agree on a common set of dealers that have distributed their inputs, but also ensure that, in the synchronous case, all honest parties are included in this set. To address this, we introduce a waiting period so that every honest party can complete input distribution in the synchronous case, and only then run agreement to determine the common set. Refer to the following detailed construction.

Protocol $\Pi_{\text{NA-MPC}}$

Let $\mathcal{P} = \{P_1, \dots, P_n\}$ denote the set of all parties, $|C|$ denote the circuit size, $|C_I|$ denote all parties' input size, and t_s, t_a be the corresponding corruption threshold when the network is synchronous or asynchronous. We use $\Pi_{\text{ba}}, \Pi_{\text{rbc}}$ to denote arbitrary network-agnostic BA and RBC protocols.

Offline Phase

Preparation of Beaver Triples. All parties start by invoking the regular procedure of $\Pi_{\text{NA-Triple}}$ with input parameter $|C| + |C_I|$ to prepare random Beaver triples. All parties will terminate $\Pi_{\text{NA-Triple}}$ with a set $\mathcal{P}'$:

- If all parties terminate the regular phase, parties in $\mathcal{P}'$ get $|C| + |C_I|$ degree- t_s random Beaver triples.
- Otherwise, all parties terminate the fallback procedure, parties in $\mathcal{P}'$ get $|C| + |C_I|$ degree- t_a random Beaver triples.

For the first $|C|$ random Beaver triples, we assign each of them to a multiplication gate. For the last $|C_I|$ Beaver triples, parties in $\mathcal{P}'$ only record the first random sharings in each triple. These random sharings will be used when parties distribute their inputs. In the following, we will use t to denote the degree of random Beaver triples that all parties prepared in the offline phase.

Online Phase

- 1: **Initialization.** All parties first invoke an instance of Π'_{ba} with input 1. Upon terminating Π'_{ba} with output 1, we know that when the network is synchronous, different honest parties will terminate Π'_{ba} at a similar time (at most 2Δ difference), where Δ is the time bound for each round in the synchronous case.
- 2: **Distributing Inputs.** For each party P_i 's input x_i , all parties assign an unused degree- t sharing $[r]_t$ for it. Then parties in $\mathcal{P}'$ send their shares of $[r]_t$ to P_i , and P_i uses online error correction to reconstruct r . Upon getting r , P_i invokes Π_{rbc} to broadcast $x_i + r$. Upon receiving $x_i + r$ from P_i , parties in $\mathcal{P}'$ locally compute $[x_i]_t = x_i + r - [r]_t$.
- 3: **Agreement on the Set of Inputs.** All parties initialize n instances of the network-agnostic Π_{ba} , one for each party P_i , denoted by $\Pi_{\text{ba}}^{(i)}$. Then, each party proceeds to the following steps after waiting for an additional $\text{Time}_{\text{bc}} + 2\Delta$ once it terminates $\Pi_{\text{ba}}^{(i)}$ in Step 1:

- (1) Once a party terminates the P_i 's Π_{rbc} , each party sets its input to $\Pi_{\text{ba}}^{(i)}$ as 1.
- (2) After a party has terminated at least $n - t_s$ instances of Π_{ba} with output 1, for all P_i that this party has not terminated P_i 's Π_{rbc} , it sets its input to $\Pi_{\text{ba}}^{(i)}$ as 0.
- (3) When all Π_{ba} instances have terminated, all parties define $\mathcal{D}$ as the set of identities corresponding to the all instances of Π_{ba} that output 1.

For every $P_i \in \mathcal{D}$, all parties hold the shares $[x_i]_t$ of its input. For every $P_i \notin \mathcal{D}$, the parties set the shares of P_i 's input to 0.

- 4: **Online Circuit Evaluation.** All parties in $\mathcal{P}'$ evaluate the circuit gate by gate:
 - For every addition gate with input sharings $[x]_t, [y]_t$, all parties in $\mathcal{P}'$ locally compute $[z]_t = [x]_t + [y]_t$.
 - For a group of at most $t_s + 1$ multiplication gates, suppose the input degree- t Shamir sharings are denoted by $([x_i]_t, [y_i]_t)_{i=1}^{t_s+1}$. Let $([a_i]_t, [b_i]_t, [c_i]_t)_{i=1}^{t_s+1}$ denote the random Beaver triples assigned to these $t_s + 1$ gates.
 1. All parties in $\mathcal{P}'$ locally compute $[x_i + a_i]_t = [x_i]_t + [a_i]_t$ and $[y_i + b_i]_t = [y_i]_t + [b_i]_t$ for all $i \in [t_s + 1]$.
 2. All parties in $\mathcal{P}'$ invoke Π_{PubRec} with their shares and the set $\mathcal{P}'$ to reconstruct $\{x_i + a_i, y_i + b_i\}_{i=1}^{t_s+1}$.
 3. For all $i \in [t_s + 1]$, all parties in $\mathcal{P}'$ locally compute:

$$[z_i]_t = (x_i + a_i)(y_i + b_i) - (x_i + a_i)[b_i]_t - (y_i + b_i)[a_i]_t + [c_i]_t.$$

- 5: **Output and Termination.** All parties do the following steps.

- (1). For all output sharings $[y]_t$, all parties in $\mathcal{P}$ invoke Π_{PubRec} with input set $\mathcal{P}'$, and parties in $\mathcal{P}'$ additionally take their shares of $[y]_t$ as inputs. All parties in $\mathcal{P}$ will eventually get the output y from Π_{PubRec} .

Analysis of the Communication Complexity. During the offline phase, parties invoke $\Pi_{\text{NA-Triple}}$ to generate triples, which costs $\mathcal{O}(((C + C_I)n + n^6) \log n)$ -bit communication. During the input phase, parties need to invoke n instances of Π_{ba} , n instances of Π_{rbc} , and one instance of Π'_{ba} , which costs $\mathcal{O}((C_I n^2 + n^5) \log n)$ -bit communication. During the evaluation and output phase, parties only need to do public reconstruction

and therefore cost $\mathcal{O}((Cn + Dn^2) \log n)$ -bit communication. Then the total communication complexity of $\Pi_{\text{NA-MPC}}$ is $\mathcal{O}((Cn + (C_I + D)n^2 + n^6) \log n)$ bits.

Lemma 1. *Protocol $\Pi_{\text{NA-MPC}}$ securely computes $\mathcal{F}_{\text{se}}$ against a fully malicious adversary $\mathcal{A}$ who corrupts at most t_s parties in the synchronous setting or t_a parties in the asynchronous setting.*

We prove Lemma 1 as follows. The construction of the ideal adversary $\mathcal{S}$ is as follows. Let $\mathcal{E}$ be the environment and $\mathcal{A}$ be the real-world adversary. $\mathcal{S}$ receives from $\mathcal{E}$ the party set $\mathcal{P} = \{P_1, \dots, P_n\}$, the set of corrupted parties Corr , and the network configuration $\text{mode} \in \{\text{synch}, \text{asynch}\}$.

Simulator $\mathcal{S}$

Let Corr denote the set of corrupted parties, $|\text{Corr}| \leq t_s$ when the network is synchronous, and $|\text{Corr}| \leq t_a$ when the network is asynchronous. We denote $\mathcal{S}_{\text{TripleIdSync}}$ and $\mathcal{S}_{\text{TripleIdAsync}}$ as the simulators used in the security proofs showing that $\Pi_{\text{Triple-Id-Sync}}$ and $\Pi_{\text{Triple-Id-Async}}$ realize $\mathcal{F}_{\text{Triple-Id-Sync}}$ and $\mathcal{F}_{\text{Triple-Id-Async}}$, respectively. At the beginning of the simulation, $\mathcal{S}$ sends the set of corrupted parties Corr and the network mode mode to $\mathcal{F}_{\text{se}}$.

Offline Phase

- 1: During the **Regular Procedure**, $\mathcal{S}$ does the following steps. Let $\mathcal{A}_{\text{TripleIdSync}}$ be the adversary for the underlying synchronous triple generation protocol $\Pi_{\text{Triple-Id-Sync}}$ who just follows the instructions received from the environment $\mathcal{E}$ and passes the messages received from honest parties in each round. $\mathcal{S}$ acts as $\mathcal{E}_{\text{TripleIdSync}}$ and $\mathcal{F}_{\text{Triple-Id-Sync}}$ and invokes $\mathcal{S}_{\text{TripleIdSync}}$.
- 2: For the simulation of $\mathcal{F}_{\text{Triple-Id-Sync}}$, $\mathcal{S}$ does the following steps.
 - (1). Receive the party set $\mathcal{P}'$ and corrupted party set Corr' from $\mathcal{S}_{\text{TripleIdSync}}$.
 - (2). Receive a set of shares $\{u_{i,j}, v_{i,j}, w_{i,j}\}_{i \in [N], j \in \text{Corr}'}$ of corrupted parties from $\mathcal{S}_{\text{TripleIdSync}}$.
 - (3). When all honest parties can get their shares of triples, simulate the behavior of sending output messages from $\mathcal{F}_{\text{Triple-Id-Sync}}$ to all honest parties in $\mathcal{P}'$. Upon receiving a request $(\text{Conflict}, P_i, P_j)$ from $\mathcal{S}$, if the output of all honest parties has not been delivered and $P_i, P_j \in \mathcal{P}'$, change it to $(\text{Conflict}, P_i, P_j)$. Otherwise, ignore this message.
- 3: In each segment of triple generation, we set $\mathcal{P}' = \mathcal{P} \setminus \mathcal{W}$, and let Corr' denote the corrupted parties in $\mathcal{P}'$. Then from $\text{rid} = 1$ to R in the current segment,
 - (1). $\mathcal{S}$ receives from $\mathcal{A}_{\text{TripleIdSync}}$ the P2P messages and broadcast messages that parties in $\mathcal{P}' \setminus \text{Corr}'$ send to parties in Corr' .
 - (2). In $\Pi_{\text{NA-Triple}}$, for each honest party $P_i \in \mathcal{P}'$, after P_i enters the ridth round, $\mathcal{S}$ prepares $(m_0^{(i)}, \{m_j^{(i)}\}_{j \notin \mathcal{W}})$ as follows: $\mathcal{S}$ sets $m_0^{(i)}$ as the broadcast messages of P_i received from $\mathcal{A}_{\text{TripleIdSync}}$. For each $P_j \in \text{Corr}'$, $\mathcal{S}$ sets $m_j^{(i)}$ as the P2P message sent from P_i to P_j received from $\mathcal{A}_{\text{TripleIdSync}}$. Then $\mathcal{S}$ takes $(m_0^{(i)}, \{m_j^{(i)}\}_{j \notin \mathcal{W}})$ as P_i 's input for Π_{Channel} and honestly execute P_i during the Π_{Channel} .
 - (3). In $\Pi_{\text{NA-Triple}}$, after all parties terminate Π_{Channel} with a set $\mathcal{W}'$, $\mathcal{S}$ follows the protocol to update $\mathcal{W}$. Then, depending on the size of the sets $\mathcal{W}'$, $\mathcal{W}$ and round id rid , if $|\mathcal{W}| \geq 2(t_s - t_a)$, $\mathcal{S}$ breaks the loop. Otherwise, $\mathcal{S}$ further checks whether $\mathcal{W}' = \emptyset$,
 - If true, then if $\text{rid} < R$, $\mathcal{S}$ moves to the next round $\text{rid} + 1$. Otherwise,
 - * If $\mathcal{S}$ has received $(\text{Conflict}, P_i, P_j)$ during the simulation of $\mathcal{F}_{\text{Triple-Id-Sync}}$, he follows the protocol to update $\mathcal{W}$ and reruns the current segment.
 - * Otherwise, $\mathcal{S}$ considers that all parties can terminate the current segment, and $\mathcal{S}$ learns corrupted parties' shares of triples in this segment. Then $\mathcal{S}$ moves to the next segment.
 - Otherwise, $\mathcal{S}$ follows the protocol to update $\mathcal{W}$ and reruns the current segment.

If $\mathcal{S}$ terminates the generation of triples for all n segments, $\mathcal{S}$ learns corrupted parties' shares of degree- t_s Beaver triples and moves to the online phase. If $\mathcal{S}$ breaks the loop, $\mathcal{S}$ moves to the fallback procedure.

- 4: When all parties move to the **Fallback Procedure**, $\mathcal{S}$ does the following steps. $\mathcal{S}$ sets $\mathcal{P}'$ as the first $4t_a + 1$ parties in $\mathcal{P} \setminus \mathcal{W}$. To simulate the fallback procedure, let $\mathcal{A}_{\text{TripleIdAsync}}$ be the adversary for the underlying asynchronous triple generation protocol $\Pi_{\text{Triple-Id-Async}}$ who follows the instructions received from the environment and passes the messages received from honest parties in each round. $\mathcal{S}$ acts as $\mathcal{E}_{\text{asynch}}$ and $\mathcal{F}_{\text{Triple-Id-Async}}$ and invokes $\mathcal{S}_{\text{TripleIdAsync}}$.
- 5: For the simulation of $\mathcal{F}_{\text{Triple-Id-Async}}$,
 - (1). Define Corr' be the set of corrupted parties in $\mathcal{P}'$.
 - (2). Receive a set of shares $\{u_{i,j}, v_{i,j}, w_{i,j}\}_{i \in [N], j \in \text{Corr}'}$ of corrupted parties from $\mathcal{S}_{\text{TripleIdAsync}}$.

- (3). When all honest parties can get their shares of triples, simulate the behavior of sending output messages from $\mathcal{F}_{\text{Triple-Id-Sync}}$ to all honest parties in $\mathcal{P}'$.
- 6: For $\mathcal{E}_{\text{asynch}}$, $\mathcal{S}$ instructs $\mathcal{A}_{\text{TripleAsynch}}$ to corrupt parties in Corr' . Then,
- Whenever $\mathcal{E}$ activates a party $P_i \in \mathcal{P}'$ in $\Pi_{\text{NA-Triple}}$, $\mathcal{S}$ activates P_i in $\Pi_{\text{Triple-Asynch}}$.
 - For each message sent from $P_i \in \text{Corr}'$ to $P_j \in \mathcal{P}' \setminus \text{Corr}'$ in $\Pi_{\text{NA-Triple}}$, $\mathcal{S}$ receives the message from $\mathcal{A}$ and instructs $\mathcal{A}_{\text{TripleAsynch}}$ to let P_i send the same message to P_j .
 - For each message sent from $P_i \in \mathcal{P}' \setminus \text{Corr}'$ to $P_j \in \text{Corr}'$ in $\Pi_{\text{Triple-Asynch}}$, $\mathcal{S}$ receives the message from $\mathcal{A}_{\text{TripleAsynch}}$ and sends it to P_j on behalf of P_i in $\Pi_{\text{Triple-Asynch}}$.
- Upon terminating $\Pi_{\text{Triple-Asynch}}$, $\mathcal{S}$ learns corrupted parties' shares of degree- t_a Beaver triples and moves to the online phase.

Online Phase

- 1: Upon terminating the offline phase, $\mathcal{S}$ learns the set $\mathcal{P}'$. We use Corr' to denote the corrupted parties set in $\mathcal{P}'$ and set $|\text{Corr}'| = t' \leq t$, where t is the degree of the random Beaver triples all parties prepared during the offline phase. Let $\mathcal{H}_C$ denote the set of first $t - t'$ honest parties in $\mathcal{P}'$.
- 2: During the **Initialization**, $\mathcal{S}$ honestly execute honest parties to participate in Π'_{ba} .
- 3: During the **Distributing Inputs**, for each party $P_i \in \mathcal{P}$:
 - If P_i is corrupted, $\mathcal{S}$ randomly samples the whole sharing $[r]_t$ based on shares of corrupted parties. Then $\mathcal{S}$ honestly executes each honest party to reconstruct r to $\mathcal{S}$. Upon receiving $x_i + r$ from P_i 's reliable broadcast, $\mathcal{S}$ extracts P_i 's inputs x_i .
 - If P_i is honest, $\mathcal{S}$ simulates the behavior of sending messages between honest parties and P_i . For each share received from an honest party, $\mathcal{S}$ considers this share to be correct. For each share received from a corrupted party, $\mathcal{S}$ checks whether it is the same as the one known to $\mathcal{S}$. If true, $\mathcal{S}$ also considers it correct. When P_i receives $2t + 1$ correct shares, $\mathcal{S}$ considers P_i succeeds in using online error correction to reconstruct the secret r .

Then, for each honest party $P_i \in \mathcal{P}$, $\mathcal{S}$ randomly samples a value as $x_i + r$ and broadcasts it on behalf of P_i .
- 4: During the **Agreement on the Set of Inputs**, $\mathcal{S}$ honestly follows the protocol to execute all instances of Π_{ba} with their inputs. Upon terminating, $\mathcal{S}$ learns the set $\mathcal{D}$ of size at least $n - t_s$. Then, $\mathcal{S}$ samples random values for each party in $\mathcal{H}_C$ as their shares of $[x_i]_t$ for each $P_i \in \mathcal{D}$. Finally, $\mathcal{S}$ sends the set $\mathcal{D}$ and x_i for all corrupted $P_i \in \mathcal{D}$ to $\mathcal{F}_{\text{sfe}}$.
- 5: During the **Online Circuit Evaluation**, for each addition gate, $\mathcal{S}$ follows the protocol to compute the output share for all parties in $\text{Corr}' \cup \mathcal{H}_C$. For multiplication gates:
 - (1). $\mathcal{S}$ follows the protocol to compute shares of $\{[x_i + a_i]_t, [y_i + b_i]_t\}_{i=1}^{t_s+1}$ for all parties in $\text{Corr}' \cup \mathcal{H}_C$.
 - (2). $\mathcal{S}$ randomly samples the whole sharings $\{[x_i + a_i]_t, [y_i + b_i]_t\}_{i=1}^{t_s+1}$ based on the shares he computed for parties in $\text{Corr}' \cup \mathcal{H}_C$. Then $\mathcal{S}$ gets each shares of $\{[x_i + a_i]_t, [y_i + b_i]_t\}_{i=1}^{t_s+1}$ for honest parties in $\mathcal{P}' \setminus \mathcal{H}_C$, and can honestly execute them during the Π_{PubRec} .
 - (3). When all parties terminate Π_{PubRec} , $\mathcal{S}$ computes the output shares for all parties in $\text{Corr}' \cup \mathcal{H}_C$.
- 6: During the **Output and Termination**, $\mathcal{S}$ sends output request to $\mathcal{F}_{\text{sfe}}$. Upon receiving $\mathbf{y}$ from $\mathcal{F}_{\text{sfe}}$, $\mathcal{S}$ computes the whole sharing $[\mathbf{y}]_t$ based on the secrets $\mathbf{y}$ and shares of $[\mathbf{y}]_t$ for parties in $\text{Corr}' \cup \mathcal{H}_C$. With the whole sharing $[\mathbf{y}]_t$, $\mathcal{S}$ honestly executes the rest of the steps. When honest parties terminate the Π_{PubRec} in the output phase, $\mathcal{S}$ delivers the output from $\mathcal{F}_{\text{sfe}}$ to this honest party.

Now we show that any computationally unbounded environment $\mathcal{E}$ cannot distinguish the real-world and the ideal-world.

Hyb₀: In the initial hybrid, we consider the real-world execution and define the output to be the output of $\mathcal{E}$.

Hyb₁: In the following, we focus on the simulation of the offline phase.

Hyb_{1.1}: In this hybrid, during the **Regular Procedure**, we change the construction of $\mathcal{S}$ as above. The only difference is that $\mathcal{S}$ learns each honest party P_i 's P2P and broadcast messages $(m_0^{(i)}, \{m_j^{(i)}\}_{j \notin \mathcal{W}})$ from $\mathcal{A}_{\text{TripleIdSync}}$. Note that here we also let $\mathcal{S}$ play the role of $\mathcal{A}_{\text{TripleIdSync}}$ during the $\Pi_{\text{Triple-Id-Sync}}$, then $\mathcal{A}_{\text{TripleIdSync}}$ will receive honest P_i 's P2P and broadcast messages generated by $\mathcal{S}_{\text{TripleIdSync}}$. Since $\Pi_{\text{Triple-Id-Sync}}$ realizes $\mathcal{F}_{\text{Triple-Id-Sync}}$ with perfect security, then the honest party P_i 's P2P and broadcast messages $\mathcal{S}$ received from $\mathcal{A}_{\text{TripleIdSync}}$ have the same distribution as in **Hyb₁**. The distributions of **Hyb_{1.1}** and **Hyb₁** are the same.

Hyb_{1.2}: In this hybrid, we let $\mathcal{S}$ play the role of $\mathcal{F}_{\text{Triple-Id-Sync}}$ during the **Regular Procedure**, and $\mathcal{S}$ will delay the delivery of honest parties' output shares until the online phase. The distributions of **Hyb_{1.2}** and **Hyb_{1.1}** are the same.

Hyb_{1.3}: In this hybrid, during the **Fallback Procedure**, we change the construction of $\mathcal{S}$ as above, and let $\mathcal{S}$ play the role of $\mathcal{F}_{\text{Triple-Async}}$. Following the same reason as in **Hyb_{1.1}**, **Hyb_{1.2}**, the distributions of **Hyb_{1.3}** and **Hyb_{1.2}** are the same.

Hyb₂: In the following, we focus on the simulation of the online phase.

Hyb_{2.1}: In this hybrid, we change the generation of random degree- t sharing $[r]_t$ assigned to an honest party in the input phase. $\mathcal{S}$ first randomly samples a value as $x_i + r$, then computes $r = x_i + r - x_i$ and the whole sharing $[r]_t$ based on the secret r and shares of corrupted parties. Since both r and $x_i + r$ are uniformly random, the distributions of **Hyb_{2.1}** and **Hyb_{2.0}** are the same.

Hyb_{2.2}: In this hybrid, let $\mathcal{H}_C$ be the set of first $t - t'$ honest parties in $\mathcal{P}'$, $\mathcal{S}$ samples random values as their shares of $[x_i]_t$ for each dealer $P_i \in \mathcal{D}$. Since the whole sharing $[x_i]_t$ is still uniformly random given shares of parties in $\text{Corr} \cup \mathcal{H}_C$, the distributions of **Hyb_{2.2}** and **Hyb_{2.1}** are the same.

Hyb_{2.3}: In this hybrid, during the **Online Circuit Evaluation**, when all parties need to reconstruct $\{[x_i + a_i]_t, [y_i + b_i]_t\}_{i=1}^{t_{s+1}}$, $\mathcal{S}$ first randomly samples the whole sharings $[x_i + a_i]_t, [y_i + b_i]_t$ based on shares of parties in $\text{Corr} \cup \mathcal{H}_C$, then compute $[a_i]_t = [x_i + a_i]_t - [x_i]_t, [b_i]_t = [y_i + b_i]_t - [y_i]_t$. The difference is that we change the generation of $[a_i]_t, [b_i]_t$, however, both of the secrets a_i, b_i and $x_i + a_i, y_i + b_i$ are sampled by $\mathcal{S}$, and they are uniformly random given shares of parties in $\text{Corr} \cup \mathcal{H}_C$. Therefore, the distributions of **Hyb_{2.3}** and **Hyb_{2.2}** are the same.

Hyb_{2.4}: In this hybrid, while reconstructing the output sharing $[y]_t$ to a corrupted receiver $\mathcal{S}$ first computes $\mathbf{y}$ from all parties' inputs and then computes $[y]_t$ from secret $\mathbf{y}$ and the shares for parties in $\text{Corr} \cup \mathcal{H}_C$. Finally, $\mathcal{S}$ honestly executes the rest of the steps on behalf of each honest party in $\mathcal{P}'$.

Here, we only change the way of generating the output shares for honest parties not in $\mathcal{H}_C$. We don't change the generation process of shares for parties in $\text{Corr} \cup \mathcal{H}_C$ of $[y]_t$ and the secret $\mathbf{y}$, and shares of $[y]_t$ for honest parties not in $\mathcal{H}_C$ are fully determined by the shares for parties in $\text{Corr} \cup \mathcal{H}_C$ and the secret. Therefore, the honest parties' shares of $[y]_t$ stay unchanged. Thus, **Hyb_{2.4}** and **Hyb_{2.3}** have the same distribution.

Hyb_{2.5}: In this hybrid, $\mathcal{S}$ doesn't generate shares for honest parties out of $\mathcal{H}_C$ of any Shamir secret sharing. For the computation of output sharings reconstructed to corrupted parties, $\mathcal{S}$ doesn't follow the protocol to compute the secrets. Instead, $\mathcal{S}$ directly uses the corrupted parties' outputs as the secrets. Clearly, by the correctness of the protocol, the outputs computed by following the protocol match the outputs from $\mathcal{F}_{\text{sfe}}$. Besides, the shares for honest parties out of $\mathcal{H}_C$ are not used in the simulation. Thus, **Hyb_{2.5}** and **Hyb_{2.4}** have the same distribution.

Hyb_{2.6}: In this hybrid, $\mathcal{S}$ no longer requires honest parties' inputs. $\mathcal{S}$ first sends the network mode mode , the set of corrupted parties Corr , the set $\mathcal{D}$ and x_i for all corrupted $P_i \in \mathcal{D}$ to $\mathcal{F}_{\text{sfe}}$. When each honest party succeeds in using online error correction to reconstruct $\mathbf{y}$ during the Π_{PubRec} in the output phase, $\mathcal{S}$ delivers the output from $\mathcal{F}_{\text{sfe}}$ to this honest party. **Hyb_{2.6}** and **Hyb_{2.5}** have the same distribution.

Note that **Hyb_{2.6}** is the ideal-world scenario, then we prove that $\Pi_{\text{NA-MPC}}$ realizes $\mathcal{F}_{\text{sfe}}$ with perfect security.

Acknowledgments. X. Ji and Y. Song were supported in part by the National Basic Research Program of China Grant 2011CBA00300, 2011CBA00301, the National Natural Science Foundation of China Grant 61033001, 61361136003. Y. Song was also supported in part by the Shanghai Qi Zhi Institute Innovation Program SQZ202313.

References

- [AAPP23] Ittai Abraham, Gilad Asharov, Shravani Patil, and Arpita Patra. Detect, pack and batch: perfectly-secure mpc with linear communication and constant expected time. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 251–281. Springer, 2023.
- [AAPP24] Ittai Abraham, Gilad Asharov, Shravani Patil, and Arpita Patra. Perfect asynchronous mpc with linear communication overhead. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 280–309. Springer, 2024.
- [AAPS25] Ittai Abraham, Gilad Asharov, Arpita Patra, and Gilad Stern. Asynchronous agreement on a core set in constant expected time and more efficient asynchronous vss and mpc. In *Theory of Cryptography Conference*, pages 451–482. Springer, 2025.
- [ABKL22] Andreea B. Alexandru, Erica Blum, Jonathan Katz, and Julian Loss. State machine replication under changing network conditions. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 681–710. Springer, Cham, December 2022.
- [AC23] Ananya Appan and Ashish Choudhury. Network agnostic MPC with statistical security. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023, Part II*, volume 14370 of *LNCS*, pages 63–93. Springer, Cham, November / December 2023.
- [ACC22] Ananya Appan, Anirudh Chandramouli, and Ashish Choudhury. Perfectly-secure synchronous MPC with asynchronous fallback guarantees. In Alessia Milani and Philipp Woelfel, editors, *41st ACM PODC*, pages 92–102. ACM, July 2022.
- [BCG93] Michael Ben-Or, Ran Canetti, and Oded Goldreich. Asynchronous secure computation. In *25th ACM STOC*, pages 52–61. ACM Press, May 1993.
- [BCLZL23] Renas Bacho, Daniel Collins, Chen-Da Liu-Zhang, and Julian Loss. Network-agnostic security comes (almost) for free in DKG and MPC. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 71–106. Springer, Cham, August 2023.
- [BCV24] Nidhish Bhimrajka, Ashish Choudhury, and Supreeth Varadarajan. Network-agnostic multi-party computation revisited (extended abstract). In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part II*, volume 14602 of *LNCS*, pages 171–204. Springer, Cham, April 2024.
- [BDD⁺21] Carsten Baum, Bernardo David, Rafael Dowsley, Jesper Buus Nielsen, and Sabine Oechsner. TARDIS: A foundation of time-lock puzzles in UC. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part III*, volume 12698 of *LNCS*, pages 429–459. Springer, Cham, October 2021.
- [Bea92] Donald Beaver. Efficient multiparty protocols using circuit randomization. In Joan Feigenbaum, editor, *Advances in Cryptology — CRYPTO ’91*, volume 576 of *Lecture Notes in Computer Science*, pages 420–432. Berlin, Heidelberg, 1992. Springer Berlin Heidelberg.
- [BGW88] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *20th ACM STOC*, pages 1–10. ACM Press, May 1988.
- [BH08] Zuzana Beerliova-Trubiniova and Martin Hirt. Perfectly-secure MPC with linear communication complexity. In Ran Canetti, editor, *Theory of Cryptography Conference — TCC 2008*, volume 4948 of *Lecture Notes in Computer Science*, pages 213–230. Springer-Verlag, 3 2008.
- [BJK⁺25a] Akhil Bandrupalli, Xiaoyu Ji, Aniket Kate, Chen-Da Liu-Zhang, Daniel Pöllmann, and Yifan Song. Velox: Scalable fair asynchronous MPC from lightweight cryptography. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 1799–1813. ACM Press, October 2025.
- [BJK⁺25b] Akhil Bandrupalli, Xiaoyu Ji, Aniket Kate, Chen-Da Liu-Zhang, and Yifan Song. Computationally efficient asynchronous MPC with linear communication and low additive overhead. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part IV*, volume 16003 of *LNCS*, pages 261–294. Springer, Cham, August 2025.
- [BKL19] Erica Blum, Jonathan Katz, and Julian Loss. Synchronous consensus with optimal asynchronous fallback guarantees. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part I*, volume 11891 of *LNCS*, pages 131–150. Springer, Cham, December 2019.

- [BKL21] Erica Blum, Jonathan Katz, and Julian Loss. Tardigrade: An atomic broadcast protocol for arbitrary network conditions. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part II*, volume 13091 of *LNCS*, pages 547–572. Springer, Cham, December 2021.
- [BKR94] Michael Ben-Or, Boaz Kelmer, and Tal Rabin. Asynchronous secure computations with optimal resilience (extended abstract). In Jim Anderson and Sam Toueg, editors, *13th ACM PODC*, pages 183–192. ACM, August 1994.
- [Bra84] Gabriel Bracha. An asynchronous $[(n - 1)/3]$ -resilient consensus protocol. In *Proceedings of the third annual ACM symposium on Principles of distributed computing*, PODC '84, page 154–162, New York, NY, USA, 1984. Association for Computing Machinery.
- [BSFO12] Eli Ben-Sasson, Serge Fehr, and Rafail Ostrovsky. Near-linear unconditionally-secure multiparty computation with a dishonest minority. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, pages 663–680, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [BZL20] Erica Blum, Chen-Da Liu Zhang, and Julian Loss. Always have a backup plan: Fully secure synchronous MPC with asynchronous fallback. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 707–731. Springer, Cham, August 2020.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
- [CCD88] David Chaum, Claude Crépeau, and Ivan Damgård. Multiparty unconditionally secure protocols (extended abstract). In *20th ACM STOC*, pages 11–19. ACM Press, May 1988.
- [CFG⁺23] Ran Cohen, Pouyan Forghani, Juan A. Garay, Rutvik Patel, and Vassilis Zikas. Concurrent asynchronous byzantine agreement in expected-constant rounds, revisited. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023, Part IV*, volume 14372 of *LNCS*, pages 422–451. Springer, Cham, November / December 2023.
- [CGHZ16] Sandro Coretti, Juan A. Garay, Martin Hirt, and Vassilis Zikas. Constant-round asynchronous multiparty computation based on one-way functions. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 998–1021. Springer, Berlin, Heidelberg, December 2016.
- [CHLZ21] Annick Chopard, Martin Hirt, and Chen-Da Liu-Zhang. On communication-efficient asynchronous MPC with adaptive security. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part II*, volume 13043 of *LNCS*, pages 35–65. Springer, Cham, November 2021.
- [Coh16] Ran Cohen. Asynchronous secure multiparty computation in constant time. In Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang, editors, *PKC 2016, Part II*, volume 9615 of *LNCS*, pages 183–207. Springer, Berlin, Heidelberg, March 2016.
- [DHLZ21] Giovanni Deligios, Martin Hirt, and Chen-Da Liu-Zhang. Round-efficient byzantine agreement and multi-party computation with asynchronous fallback. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part I*, volume 13042 of *LNCS*, pages 623–653. Springer, Cham, November 2021.
- [DLZ23] Giovanni Deligios and Chen-Da Liu-Zhang. Synchronous perfectly secure message transmission with optimal asynchronous fallback guarantees. In Foteini Baldimtsi and Christian Cachin, editors, *FC 2023, Part I*, volume 13950 of *LNCS*, pages 77–93. Springer, Cham, May 2023.
- [DME24] Giovanni Deligios and Mose Mizrahi Erbes. Closing the efficiency gap between synchronous and network-agnostic consensus. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 432–461. Springer, 2024.
- [DN07] Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In *Advances in Cryptology - CRYPTO 2007, 27th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2007, Proceedings*, volume 4622 of *Lecture Notes in Computer Science*, pages 572–590. Springer, 2007.
- [GLS19] Vipul Goyal, Yanyi Liu, and Yifan Song. Communication-efficient unconditional mpc with guaranteed output delivery. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019*, pages 85–114, Cham, 2019. Springer International Publishing.
- [GLZS24] Vipul Goyal, Chen-Da Liu-Zhang, and Yifan Song. Towards achieving asynchronous MPC with linear communication and optimal resilience. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VIII*, volume 14927 of *LNCS*, pages 170–206. Springer, Cham, August 2024.
- [GLZW22] Diana Ghinea, Chen-Da Liu-Zhang, and Roger Wattenhofer. Optimal synchronous approximate agreement with asynchronous fallback. In Alessia Milani and Philipp Woelfel, editors, *41st ACM PODC*, pages 70–80. ACM, July 2022.
- [GLZW23] Diana Ghinea, Chen-Da Liu-Zhang, and Roger Wattenhofer. Multidimensional approximate agreement with asynchronous fallback. In *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures*, pages 141–151, 2023.

- [GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th ACM STOC*, pages 218–229. ACM Press, May 1987.
- [GSZ20] Vipul Goyal, Yifan Song, and Chenzhi Zhu. Guaranteed output delivery comes free in honest majority MPC. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 618–646. Springer, Cham, August 2020.
- [HMP00] Martin Hirt, Ueli Maurer, and Bartosz Przydatek. Efficient secure multi-party computation. In *International conference on the theory and application of cryptology and information security*, pages 143–161. Springer, 2000.
- [HNP05] Martin Hirt, Jesper Buus Nielsen, and Bartosz Przydatek. Cryptographic asynchronous multi-party computation with optimal resilience (extended abstract). In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 322–340. Springer, Berlin, Heidelberg, May 2005.
- [JLZPS26] Xiaoyu Ji, Chen-Da Liu-Zhang, Daniel Pöllmann, and Yifan Song. Information-theoretic network-agnostic mpc with polynomial communication. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2026.
- [KMTZ13] Jonathan Katz, Ueli Maurer, Björn Tackmann, and Vassilis Zikas. Universally composable synchronous computation. In Amit Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 477–498. Springer, Berlin, Heidelberg, March 2013.
- [LZLM⁺19] Chen-Da Liu-Zhang, Julian Loss, Ueli Maurer, Tal Moran, and Daniel Tschudi. Robust MPC: Asynchronous responsiveness yet synchronous security. Cryptology ePrint Archive, Report 2019/159, 2019.
- [PP24] Shravani Patil and Arpita Patra. Perfectly-secure network-agnostic MPC with optimal resiliency. Cryptology ePrint Archive, Paper 2024/990, 2024.
- [RB89] Tal Rabin and Michael Ben-Or. Verifiable secret sharing and multiparty protocols with honest majority (extended abstract). In *21st ACM STOC*, pages 73–85. ACM Press, May 1989.
- [Yao82] Andrew Chi-Chih Yao. Theory and applications of trapdoor functions (extended abstract). In *23rd FOCS*, pages 80–91. IEEE Computer Society Press, November 1982.

A Review of the Triple Generation in [BH08]

In this section, we review the triple generation protocol in [BH08] that achieves security with identifiable abort. We use d to denote the degree of triples to be generated, n to denote the number of parties, and t to denote the corruption threshold among these n parties. The parameters d, n, t satisfy that $t \leq d$ and $d + 2t < n$.

At a high level, there are three main stages for their construction.

- **Stage 1: Generating Non-robust Triples.**
- **Stage 2: Agreement on the Success of Triple Generation.**
- **Stage 3: Fault Localization.**

Realization of Stage 1. Parties first prepare random double sharings in the following form.

$$([a]_d, [a]_t), ([b]_d, [b]_t), ([r]_d, [r]_{2t})$$

The authors in [BH08] show how this can be realized with the help of the hyper-invertible matrix, and we refer to the details of this part to their original paper [BH08]. During the generation of these sharings, parties need to verify whether the degree of each sharing is correct and the secrets in each pair of double sharing are the same. If not, at least one honest party would be “unhappy”, indicating the generation of triples fails for this party.

Then all parties use these double sharings to generate a triple in the form of $([a]_d, [b]_d, [c]_d)$, where the first two sharings come from their prepared double sharings, and the goal is to compute the third sharing $[c]_d$ such that $c = a \cdot b$. To achieve this goal, all parties do the following steps.

1. Parties locally compute their shares of $[c]_{2t} = [a]_t \cdot [b]_t$. Then they compute $[e]_{2t} = [c]_{2t} - [r]_{2t}$. Note that if a party is “unhappy” with their shares of the double sharings, it will set its share of $[e]_{2t}$ to be $\perp$.

2. Parties exchange their shares of $[e]_{2t}$ to reconstruct the secret e . During this procedure, if a party receives $\perp$ from any party, it will also be “unhappy”. If the received n shares do not lie on a unique degree- $2t$ polynomial, parties also become “unhappy”.
3. Upon getting e , parties compute their shares of $[c]_d := [r]_d + e$.

As a result, upon terminating the triple generation protocol, we guarantee that parties can either get their shares of $([a]_d, [b]_d, [c]_d)$ or be “unhappy”. Then the next step is to let all parties identify whether all of them are “happy”. If true, this means all parties have got their shares of triples, and they can terminate. Otherwise, they need to detect a pair of parties that are responsible for the failure of the triple generation process.

In [BH08], the reconstruction of $[e]_{2t}$ is optimized via batch reconstruction that achieves linear communication.

Realization of Stage 2. In this stage, each party P_i that has got its shares of triples will broadcast (P_i, happy) , and $(P_i, \text{unhappy})$ otherwise. Then parties wait to receive all these broadcast messages. If all of them are in the form of (P_i, happy) , then they can terminate with their shares of triples. Otherwise, this means at least one party P_i considers the triple generation fails, and parties need to do fault localization and detect the reason why P_i is “unhappy”.

We note that in the original construction in [BH08], the authors let all parties first exchange their happy bit with each other and then invoke an instance of the BA protocol to reach an agreement. This only costs $\mathcal{O}(n^2 \log n)$ -bit communication. However, the BA protocol would require $\mathcal{O}(n)$ rounds to terminate. Here we use n instances of broadcast to replace the BA protocol. Although the communication complexity would blow up to $\mathcal{O}(n^3 \log n)$ bits, the round complexity is reduced to a constant since the broadcast protocol can be realized within constant rounds.

Realization of Stage 3. In this stage, parties proceed with the following steps.

1. Parties first select a party P_r with the smallest index r as the referee, then they send all their views to P_r .
2. With all parties’ views, P_r can locally finish all computation for the triple generation procedure. As a result, P_r can detect that there exists a pair of parties (P_i, P_j) , and a message with index ℓ where P_i should have sent x to P_j , but P_j claims to have received $x' \neq x$. Then P_r broadcast (ℓ, P_i, P_j, x, x') .
3. Upon receiving (ℓ, P_i, P_j, x, x') from P_r , the accused parties P_i and P_j need to broadcast whether they agree with P_r . Let E denote the pair of parties to be eliminated. If P_i disagrees, set $E = \{P_r, P_i\}$. If P_j disagrees, set $E = \{P_r, P_j\}$. Otherwise, set $E = \{P_i, P_j\}$.

For the eliminated parties in E , the authors in [BH08] show that at least one of them is corrupted. Refer to the original paper for more details. During this procedure, each party needs to broadcast $\mathcal{O}(\log n)$ bits, and P_r needs to broadcast $\mathcal{O}(|\ell| + \log n)$ bits.